API Automation

Complete Interview Guide

Kalpesh Jain

SDET @ Algebrik AI | Mentor | 42K+ Community

Bengaluru, Karnataka, India

linkedin.com/in/kalpeshnjain09

JSPM University Pune

API Fundamentals	HTTP Methods	Status Codes	Request & Response
Headers & Auth	API Validation	API Chaining	Postman
Rest Assured	Data-Driven Testing	CI/CD Integration	Schema Validation
API Mocking	Real-Time Scenarios	Best Practices	

Table of Contents

2. What is Response
3. Request Components ★
4. Response Components ★
5. Request Body
6. Response Body
7. Query Parameters
8. Path Parameters ★
9. Real-Time Flow
10. Best Practices

HEADERS & AUTHENTICATION

1. What are Headers
2. Types of Headers
3. Authentication Basics ★
4. Bearer Token Authentication ★
5. Basic Authentication
6. API Keys
7. OAuth Basics ★
8. Real-Time Scenarios
9. Security Best Practices
10. Common Interview Questions

API VALIDATION

1. What is API Validation
2. Status Code Validation ★
3. Response Body Validation ★
4. Header Validation
5. Schema Validation ★
6. Response Time Validation
7. Data Validation
8. Negative Validation ★
9. Real-Time Scenarios
10. Best Practices

API CHAINING

1. What is API Chaining
2. Why API Chaining is Used ★
3. Dynamic Data Handling
4. Token Chaining ★
5. ID Chaining ★
6. Multi-Step API Flow
7. Real-Time Scenarios ★
8. Common Challenges
9. Best Practices
10. Common Interview Questions

POSTMAN

1. What is Postman
2. Postman Features ★
3. Collections
4. Environment Variables ★
5. Request Creation
6. Response Validation
7. Authorization Handling
8. Pre-request Scripts
9. Tests Tab ★
10. Collection Runner
11. Newman CLI ★
12. Real-Time Scenarios
13. Best Practices

REST ASSURED

1. What is Rest Assured
2. Rest Assured Architecture
3. HTTP Methods in Rest Assured ★
4. Request Specification ★
5. Response Validation ★
6. Authentication Handling

7. JSON Path Extraction ★
8. Serialization & Deserialization ★
9. API Chaining
10. Framework Integration
11. Real-Time Scenarios
12. Best Practices

DATA-DRIVEN API TESTING

1. What is Data-Driven Testing
2. Why Data-Driven Testing is Used ★
3. Types of Test Data
4. JSON Data Handling ★
5. Excel Data Handling
6. CSV Data Handling
7. Dynamic Data Generation ★
8. Parameterization ★
9. Real-Time Scenarios
10. Framework Integration
11. Best Practices
12. Common Interview Questions

CI/CD INTEGRATION (JENKINS)

1. What is CI/CD
2. What is Jenkins ★
3. Why Jenkins is Used
4. Jenkins Pipeline Basics ★
5. Integrating API Automation with Jenkins ★
6. Maven Integration
7. Newman Integration
8. Scheduled Execution
9. Reporting in Jenkins ★
10. Real-Time Scenarios
11. Best Practices
12. Common Interview Questions

SCHEMA VALIDATION

1. What is Schema Validation
2. Why Schema Validation is Important ★
3. JSON Schema Basics
4. Response Structure Validation
5. Required Fields Validation ★
6. Data Type Validation
7. Nested JSON Validation ★
8. Schema Validation in Rest Assured ★
9. Schema Validation in Postman
10. Real-Time Scenarios
11. Best Practices
12. Common Interview Questions

API MOCKING

1. What is API Mocking
2. Why API Mocking is Important ★
3. Mock Server Basics
4. Types of Mocking
5. Mocking in Postman ★
6. Mocking Using WireMock ★
7. Mocking Third-Party APIs
8. Simulating Error Responses ★
9. Real-Time Scenarios ★
10. Best Practices
11. Common Mistakes
12. Common Interview Questions

REAL-TIME SCENARIOS

1. What are Real-Time Scenarios
2. Login API Automation ★
3. User Management APIs
4. E-Commerce API Flow ★
5. Banking API Flow

6. Payment Gateway APIs ★
7. File Upload APIs
8. OTP & Authentication APIs
9. Microservices API Testing ★
10. Error Handling Scenarios
11. Best Practices
12. Common Interview Questions

BEST PRACTICES

1. What are API Automation Best Practices
2. Reusable Framework Design ★
3. Strong Assertions ★
4. Dynamic Test Data Handling
5. Proper API Validation
6. Negative Testing ★
7. Environment Management
8. Logging & Reporting
9. CI/CD Integration
10. Error Handling & Retry Logic ★
11. Test Data Cleanup
12. Common Interview Questions

API FUNDAMENTALS

1. What is API

What is it

API (Application Programming Interface) is a medium that allows two applications or systems to communicate with each other.

Key Components

- Request
- Response
- Endpoint
- Client
- Server

How YOU should answer

API acts as a bridge between client and server. It allows applications to exchange data using requests and responses without directly accessing internal implementation.

Practical Example

```
Example

Mobile App → API → Database

Example: PhonePe app sends payment request to backend API.
```

Common Mistakes

- ✗ Saying API is only for testing
- ✗ Confusing API with database
- ✗ Ignoring request-response flow

Expected Interview Questions

- ■ What is API?
- ■ Why are APIs used?
- ■ Real-time API example?

---

2. Types of APIs

What is it

Different API categories based on accessibility and architecture.

Key Components

- REST API
- SOAP API
- GraphQL API
- Internal API
- Public API

How YOU should answer

REST APIs are most commonly used in modern applications because they are lightweight and use JSON for communication. SOAP APIs are XML-based and more strict.

Practical Example

```
Example
REST API → JSON
SOAP API → XML
```

Common Mistakes

- ✕ Not knowing REST vs SOAP
- ✕ Saying SOAP is modern standard
- ✕ Ignoring GraphQL basics

Expected Interview Questions

- ■ Difference between REST and SOAP?
- ■ What APIs have you worked on?
- ■ Why is REST popular?

---

3. API Architecture

What is it

API architecture defines how APIs are designed and communicate.

Key Components

- Client
- Server
- Endpoint

- HTTP Protocol
- Request/Response

How YOU should answer

API architecture follows a client-server model where the client sends requests and the server processes and returns responses over HTTP protocol.

Practical Example

```
Example
Client → HTTP Request → Server
Server → HTTP Response → Client
```

Common Mistakes

- ✕ Confusing frontend and backend roles
- ✕ Not understanding stateless communication

Expected Interview Questions

- ■ Explain API architecture.
- ■ What is client-server communication?
- ■ What protocol do APIs use?

---

4. Client-Server Communication

What is it

Communication model where client sends requests and server returns responses.

Key Components

- Request
- Response
- HTTP
- Status Code
- Data Exchange

How YOU should answer

In client-server communication, the client sends HTTP requests to the server, and the server processes the request and returns a response with status code and data.

Practical Example

```
Example
Browser → Login Request → Server
Server → Authentication Response → Browser
```

Common Mistakes

- ✕ Ignoring status codes
- ✕ Confusing request and response

Expected Interview Questions

- ■ Explain request-response cycle.
- ■ What happens when API is called?
- ■ What is stateless communication?

---

5. REST API Basics ★

What is it

REST (Representational State Transfer) is a lightweight architecture used for API communication over HTTP.

Key Components

- • HTTP methods
- • Stateless
- • JSON
- • Resource-based URLs

How YOU should answer

REST APIs use HTTP methods like GET, POST, PUT, and DELETE for communication. They are lightweight, stateless, and mostly use JSON data format.

Practical Example

```
Example
GET /users
POST /users
PUT /users/1
DELETE /users/1
```

Common Mistakes

- ✕ Not understanding statelessness

- ✕ Confusing REST with HTTP
- ✕ Ignoring resource-based design

Expected Interview Questions

- ■ What is REST API?
- ■ Why is REST widely used?
- ■ What is a stateless API?

---

6. JSON Basics

What is it

JSON (JavaScript Object Notation) is a lightweight data format used in APIs.

Key Components

- • Key-value pairs
- • Objects
- • Arrays
- • Nested JSON

How YOU should answer

JSON is a lightweight and readable data format used for request and response communication in REST APIs.

Practical Example

```
Example
{
"name": "Kalpesh",
"role": "SDET"
}
```

Common Mistakes

- ✕ Invalid JSON syntax
- ✕ Confusing object and array

Expected Interview Questions

- ■ What is JSON?
- ■ Why is JSON preferred?
- ■ Difference between JSON object and array?

---

7. Real-Time API Flow ★

What is it

Understanding how APIs work in real applications.

Key Components

- Frontend
- Backend
- Database
- API layer

How YOU should answer

In real-time applications, frontend sends requests to backend APIs, APIs process business logic, interact with database, and return responses to frontend.

Practical Example

```
Example
Mobile App
↓
API Layer
↓
Database
```

Common Mistakes

- ✕ Thinking frontend directly accesses database
- ✕ Ignoring backend processing

Expected Interview Questions

- ■ Explain real-time API flow.
- ■ How does frontend communicate with backend?
- ■ Where do APIs fit in architecture?

---

8. Best Practices

What is it

Guidelines for understanding and testing APIs effectively.

Key Components

- Proper validations
- Understanding architecture
- Response verification
- Error handling

How YOU should answer

I focus on understanding request-response flow, validating responses correctly, and using proper API structures for maintainable testing.

Practical Example

```
Validate
- Status Code
- Response Body
- Headers
- Response Time
```

Common Mistakes

- ✕ Only validating status codes
- ✕ Ignoring negative testing
- ✕ Weak validations

Expected Interview Questions

- ■ API testing best practices?
- ■ What validations do you perform?
- ■ How do you test APIs effectively?

★ Most Important Topics

1. REST API
2. Request-Response Flow
3. JSON Basics
4. Real-Time Architecture

HTTP METHODS

1. GET Method

What is it

GET method is used to retrieve/fetch data from the server.

Key Components

- Read operation
- No request body
- Query parameters
- Safe operation

How YOU should answer

GET method is used to fetch data from the server without modifying resources. It is commonly used for retrieving user details, reports, or records.

Practical Example

```
Example
GET /users
GET /users/101
GET /products?category=mobile
```

Common Mistakes

- ✕ Sending body in GET request
- ✕ Using GET for update operations
- ✕ Ignoring query parameters

Expected Interview Questions

- ■ What is the GET method?
- ■ Can GET modify data?
- ■ Difference between GET and POST?

---

2. POST Method ★

What is it

POST method is used to create new resources/data on the server.

Key Components

- Create operation
- Request body
- JSON payload
- Data submission

How YOU should answer

POST method is used to send data to the server for creating resources such as users, orders, or transactions.

Practical Example

```
Example
POST /users
Content-Type: application/json

{ "name": "Kalpesh", "role": "SDET" }
```

Common Mistakes

- ✕ Missing request body
- ✕ Invalid JSON format
- ✕ Not validating response

Expected Interview Questions

- ■ What is POST method?
- ■ Why POST is not idempotent?
- ■ Difference between POST and PUT?

---

3. PUT Method ★

What is it

PUT method is used to update existing resources completely.

Key Components

- Update operation
- Full resource update
- Idempotent operation

How YOU should answer

The PUT method updates an existing resource completely. Sending the same PUT request multiple times produces the same result.

Practical Example

```
Example
PUT /users/1

{ "name": "Kalpesh Updated", "role": "SDET" }
```

Common Mistakes

- ✕ Using PUT for partial updates
- ✕ Confusing PUT and PATCH

Expected Interview Questions

- ■ Difference between PUT and POST?
- ■ Why PUT is idempotent?
- ■ Real-time PUT example?

---

4. DELETE Method

What is it

DELETE method removes resources/data from the server.

Key Components

- • Delete operation
- • Resource removal
- • Permanent deletion

How YOU should answer

The DELETE method removes existing resources from the server such as deleting users, records, or transactions.

Practical Example

```
Example
DELETE /users/1
```

Common Mistakes

- ✕ Deleting wrong resources
- ✕ Ignoring authorization validation

Expected Interview Questions

- ■ What is the DELETE method?
- ■ How do you validate delete operations?
- ■ Real-time delete scenario?

---

5. PATCH Method

What is it

PATCH method is used for partial updates of resources.

Key Components

- • Partial update
- • Lightweight update
- • Specific field modification

How YOU should answer

The PATCH method updates only specific fields of a resource instead of replacing the entire object.

Practical Example

```
Example
PATCH /users/1

{ "role": "Senior SDET" }
```

Common Mistakes

- ✕ Confusing PATCH with PUT
- ✕ Sending full payload unnecessarily

Expected Interview Questions

- ■ Difference between PUT and PATCH?
- ■ Why is PATCH useful?
- ■ Real-time PATCH example?

---

6. Idempotency ★

What is it

Idempotency means performing the same operation multiple times gives the same result.

Key Components

- Repeatable operation
- Same result
- Resource consistency

How YOU should answer

Idempotent APIs return the same result even if the same request is executed multiple times. GET, PUT, and DELETE are generally idempotent, while POST is not.

Practical Example

```
Example
Sending same request multiple times:
→ Same final state
```

Common Mistakes

- ✕ Saying POST is idempotent
- ✕ Confusing response and state

Expected Interview Questions

- ■ What is idempotency?
- ■ Which HTTP methods are idempotent?
- ■ Why is POST not idempotent?

---

7. Real-Time Scenarios ★

1. User Registration

POST /users

2. Fetch User Details

GET /users/1

3. Update Profile

PUT /users/1

4. Delete Account

DELETE /users/1

---

8. Best Practices

What is it

Guidelines for using HTTP methods properly in APIs.

Key Components

- Correct method usage
- Proper validations
- Resource-based APIs
- Secure operations

How YOU should answer

I ensure correct HTTP method usage, proper validations, and meaningful assertions to maintain API reliability and clarity.

Common Mistakes

- ✕ Wrong HTTP method usage
- ✕ Ignoring status code validations
- ✕ Weak API design understanding

Expected Interview Questions

- ■ Explain all HTTP methods.
- ■ Which methods are idempotent?
- ■ Difference between PUT and PATCH?

★ Most Important Topics

1. POST vs PUT
2. Idempotency
3. PUT vs PATCH
4. Real-time CRUD operations

STATUS CODES

1. What are Status Codes

What is it

HTTP status codes are server responses that indicate whether an API request was successful or failed.

Key Components

- Response status
- Success codes
- Error codes
- Client/server communication

How YOU should answer

Status codes are HTTP response indicators that tell whether an API request succeeded, failed, or encountered an error.

Practical Example

```
Example
200 → Success
404 → Not Found
500 → Server Error
```

Common Mistakes

- ✕ Ignoring status code validation
- ✕ Validating only response body
- ✕ Confusing client and server errors

Expected Interview Questions

- ■ What are HTTP status codes?
- ■ Why validate status codes?
- ■ Commonly used status codes?

---

2. 2xx Success Codes ★

What is it

2xx codes indicate successful API operations.

Key Components

- 200 OK
- 201 Created
- 204 No Content

How YOU should answer

2xx status codes indicate successful request processing. 200 means success, 201 means resource created, and 204 means success with no response body.

Practical Example

```
Example
200 OK → Fetch successful
201 Created → User created
204 No Content → Delete successful
```

Common Mistakes

- ✕ Expecting 200 for every API
- ✕ Ignoring 201 for create operations

Expected Interview Questions

- ■ Difference between 200 and 201?
- ■ When do you get 204?
- ■ Which success codes have you used?

---

3. 4xx Client Error Codes ★

What is it

4xx codes indicate client-side/request-related issues.

Key Components

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found

How YOU should answer

4xx status codes occur when the request sent by client is invalid, unauthorized, or requests unavailable resources.

Practical Example

```
Example
400 → Invalid request
401 → Authentication missing
403 → Access denied
404 → Resource not found
```

Common Mistakes

- ✕ Confusing 401 and 403
- ✕ Ignoring negative testing

Expected Interview Questions

- ■ Difference between 401 and 403?
- ■ What causes 400 Bad Request?
- ■ Real-time 404 scenario?

---

4. 5xx Server Error Codes ★

What is it

5xx codes indicate server-side failures.

Key Components

- • 500 Internal Server Error
- • 502 Bad Gateway
- • 503 Service Unavailable

How YOU should answer

5xx status codes indicate backend or server-side issues where the request reached server successfully but processing failed.

Practical Example

```
Example
500 → Internal server failure
503 → Service unavailable
```

Common Mistakes

- ✕ Assuming all failures are frontend issues
- ✕ Not reporting server logs properly

Expected Interview Questions

- What is 500 Internal Server Error?
- Difference between 4xx and 5xx?
- How do you debug server failures?

---

5. Most Important Status Codes ★

Commonly Used Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

How YOU should answer

The most commonly used status codes in API testing are 200, 201, 400, 401, 403, 404, and 500.

Expected Interview Questions

- Most common status codes?
- Which codes do you validate most?
- Difference between success and error codes?

---

6. Real-Time Scenarios ★

1. Login Failure

401 Unauthorized

2. Invalid API Payload

400 Bad Request

3. Resource Not Found

404 Not Found

4. Backend Crash

500 Internal Server Error

---

7. Negative Testing ★

What is it

Testing APIs with invalid or unexpected data.

Key Components

- Invalid payload
- Missing headers
- Unauthorized access
- Boundary testing

How YOU should answer

Negative testing validates how APIs behave with invalid inputs, unauthorized access, or incorrect requests to ensure proper error handling.

Practical Example

```
Example

Missing token → 401

Invalid endpoint → 404
```

Common Mistakes

- ✕ Testing only positive scenarios
- ✕ Ignoring invalid data testing

Expected Interview Questions

- ■ What is negative testing?
- ■ Why is negative testing important?
- ■ Examples of API negative scenarios?

---

8. Best Practices

What is it

Guidelines for validating and handling API status codes effectively.

Key Components

- Validate all responses
- Include negative testing
- Log failures properly
- Verify error messages

How YOU should answer

I validate both success and failure status codes, perform negative testing, and verify meaningful error responses for reliable API validation.

Common Mistakes

- ✕ Validating only success cases
- ✕ Ignoring error response body
- ✕ Weak assertions

Expected Interview Questions

- ■ How do you validate APIs?
- ■ What negative scenarios do you test?
- ■ Why are status codes important?

★ Most Important Topics

1. 401 vs 403
2. 4xx vs 5xx
3. Negative Testing
4. 200 vs 201

REQUEST & RESPONSE

1. What is Request

What is it

A request is data sent from client to server asking for some operation or information.

Key Components

- Endpoint
- Method
- Headers
- Body
- Parameters

How YOU should answer

An API request is sent by the client to a server containing operation details such as endpoint, HTTP method, headers, and request data.

Practical Example

```
Example
POST /users
```

Common Mistakes

- ✕ Missing mandatory headers
- ✕ Invalid payload structure
- ✕ Wrong endpoint usage

Expected Interview Questions

- ■ What is API request?
- ■ What are request components?
- ■ Difference between request and response?

---

2. What is Response

What is it

A response is data returned by server after processing the client request.

Key Components

- Status code
- Response body
- Headers
- Response time

How YOU should answer

API response contains server output after processing a request, including status code, response body, and headers.

Practical Example

```
Example
{
"id": 1,
"name": "Kalpesh"
}
```

Common Mistakes

- ✕ Validating only status code
- ✕ Ignoring response body validation

Expected Interview Questions

- ■ What is API response?
- ■ What validations do you perform on response?
- ■ What does response contain?

---

3. Request Components ★

What is it

Important elements included in API requests.

Key Components

- URL
- HTTP Method
- Headers
- Body
- Authentication

- Parameters

How YOU should answer

API requests contain endpoint URL, HTTP method, headers, authentication details, parameters, and optional request body.

Practical Example

```
Example
POST /users
Authorization: Bearer token
Content-Type: application/json
```

Common Mistakes

- ✕ Missing authentication
- ✕ Wrong content type
- ✕ Invalid parameters

Expected Interview Questions

- ■ Explain request structure.
- ■ What are request headers?
- ■ What is request payload?

---

4. Response Components ★

What is it

Important data returned in API responses.

Key Components

- Status code
- Response body
- Headers
- Cookies
- Response time

How YOU should answer

API responses contain status code, response body, headers, and performance information used for validations.

Practical Example

```
Example
200 OK
Content-Type: application/json
```

Common Mistakes

- ✕ Ignoring headers validation
- ✕ Not validating response time

Expected Interview Questions

- ■ Explain response structure.
- ■ What validations do you perform?
- ■ Why validate headers?

---

5. Request Body ★

What is it

Request body contains data sent to server during API operations.

Key Components

- • JSON payload
- • Input data
- • Objects
- • Arrays

How YOU should answer

Request body contains input data sent to API for operations like create or update actions.

Practical Example

```
Example
{
"name": "Kalpesh",
"role": "SDET"
}
```

Common Mistakes

- ✕ Invalid JSON format

- ✕ Missing mandatory fields
- ✕ Wrong data types

Expected Interview Questions

- ■ What is request payload?
- ■ When is request body used?
- ■ Can GET have request body?

---

6. Response Body ★

What is it

Response body contains data returned from server.

Key Components

- • JSON response
- • API output
- • Business data
- • Error message

How YOU should answer

Response body contains API output data returned from server after request processing.

Practical Example

```
Example
{
"status": "success",
"userId": 101
}
```

Common Mistakes

- ✕ Weak field validations
- ✕ Ignoring error messages

Expected Interview Questions

- ■ How do you validate response body?
- ■ What validations do you perform?
- ■ Difference between response body and headers?

---

7. Query Parameters

What is it

Query parameters are optional key-value pairs passed in URL for filtering or searching data.

Key Components

- Filtering
- Searching
- Pagination

How YOU should answer

Query parameters are used for filtering or customizing API responses without changing endpoint structure.

Practical Example

```
Example
GET /users?page=1
GET /products?category=mobile
```

Common Mistakes

- ✕ Invalid query format
- ✕ Hardcoded query values

Expected Interview Questions

- ■ What are query parameters?
- ■ Difference between path and query parameters?
- ■ Real-time example?

---

8. Path Parameters ★

What is it

Path parameters identify specific resources directly in URL path.

Key Components

- Resource identification
- Dynamic values
- URL path

How YOU should answer

Path parameters are dynamic values in API URLs used to identify specific resources.

Practical Example

```
Example
GET /users/101
```

Common Mistakes

- ✕ Confusing path and query params
- ✕ Hardcoding resource IDs

Expected Interview Questions

- ■ What are path parameters?
- ■ Difference between query and path params?
- ■ Real-time path param example?

---

9. Real-Time API Flow ★

1. User Login Flow

Client → Login Request → Server → Token Response → Client

2. Product Search

GET /products?category=mobile

3. Fetch User Profile

GET /users/101

---

10. Best Practices

What is it

Guidelines for validating requests and responses effectively.

Key Components

- • Proper validations
- • Secure headers
- • Validate body and status
- • Negative testing

How YOU should answer

I validate request structure, response body, headers, status codes, and error messages to ensure reliable API behavior.

Common Mistakes

- ✕ Validating only response code
- ✕ Ignoring headers and body
- ✕ No negative testing

Expected Interview Questions

- ■ What validations do you perform?
- ■ What is request payload?
- ■ Difference between path and query parameters?

★ Most Important Topics

1. Request vs Response
2. Path vs Query Parameters
3. Request Body vs Response Body
4. Real-Time API Flow

HEADERS & AUTHENTICATION

1. What are Headers

What is it

Headers are key-value pairs sent between client and server containing metadata about the request or response.

Key Components

- Authorization
- Content-Type
- Accept
- Cookies
- Custom headers

How YOU should answer

Headers provide additional information about API requests and responses such as authentication, content type, and communication preferences.

Practical Example

```
Example
Content-Type: application/json
Authorization: Bearer token
```

Common Mistakes

- ✕ Missing mandatory headers
- ✕ Wrong content type
- ✕ Exposing sensitive data

Expected Interview Questions

- ■ What are headers?
- ■ Why are headers important?
- ■ Common headers used in APIs?

---

2. Types of Headers

What is it

Different categories of headers used during API communication.

Key Components

- Request headers
- Response headers
- General headers
- Security headers

How YOU should answer

Request headers are sent from client to server, while response headers are returned by server to provide metadata about the response.

Practical Example

```
Example
Request Header:
Authorization: Bearer token

Response Header:
Content-Type: application/json
```

Common Mistakes

- ✕ Confusing request and response headers
- ✕ Ignoring response header validations

Expected Interview Questions

- ■ Difference between request and response headers?
- ■ Which headers have you validated?
- ■ Why validate Content-Type?

---

3. Authentication Basics ★

What is it

Authentication verifies user or system identity before granting API access.

Key Components

- Username/password
- Tokens

- API keys
- OAuth

How YOU should answer

Authentication ensures only authorized users or systems can access secured APIs using credentials, tokens, or authentication mechanisms.

Practical Example

```
Example
Login API → Token Generated → Access Secured APIs
```

Common Mistakes

- ✕ Exposing credentials in code
- ✕ Ignoring token expiration
- ✕ Weak security practices

Expected Interview Questions

- ■ What is API authentication?
- ■ Why is authentication important?
- ■ Types of API authentication?

---

4. Bearer Token Authentication ★

What is it

Bearer token authentication uses tokens passed in Authorization header to access protected APIs.

Key Components

- JWT token
- Authorization header
- Token expiration
- Secure access

How YOU should answer

Bearer token authentication is widely used in REST APIs where tokens are passed in Authorization headers for secure API access.

Practical Example

```
Example
```

```
Authorization: Bearer eyJhbGci...
```

Common Mistakes

- ✕ Hardcoding tokens
- ✕ Using expired tokens
- ✕ Incorrect token format

Expected Interview Questions

- ■ What is bearer token?
- ■ How do you handle token authentication?
- ■ Difference between session and token auth?

---

5. Basic Authentication

What is it

Basic authentication uses username and password encoded in request headers.

Key Components

- • Username
- • Password
- • Base64 encoding

How YOU should answer

Basic authentication sends encoded username and password in headers for authentication, but it is less secure compared to token-based authentication.

Practical Example

```
Example
Authorization: Basic dXNlcjpwYXNz
```

Common Mistakes

- ✕ Sending credentials without HTTPS
- ✕ Confusing encoding with encryption

Expected Interview Questions

- ■ What is basic authentication?
- ■ Why bearer token is preferred?
- ■ Difference between encoding and encryption?

---

6. API Keys

What is it

API keys are unique identifiers used to authenticate API consumers.

Key Components

- API key
- Access control
- Usage tracking

How YOU should answer

API keys are unique identifiers used for authentication and tracking API usage by applications or users.

Practical Example

```
Example
x-api-key: abc123xyz
```

Common Mistakes

- ✕ Exposing API keys publicly
- ✕ Hardcoding keys in repository

Expected Interview Questions

- ■ What are API keys?
- ■ Difference between API key and token?
- ■ Why secure API keys?

---

7. OAuth Basics ★

What is it

OAuth is an authorization framework that allows secure third-party access without exposing passwords.

Key Components

- Access token
- Authorization server
- Client application
- Resource owner

How YOU should answer

OAuth allows secure authorization between applications using tokens instead of sharing user credentials directly.

Practical Example

```
Example
Login with Google → OAuth → Access Token
```

Common Mistakes

- ✕ Confusing OAuth with authentication only
- ✕ Not understanding token flow

Expected Interview Questions

- ■ What is OAuth?
- ■ Why OAuth is used?
- ■ Difference between OAuth and basic auth?

---

8. Real-Time Scenarios ★

1. Login Flow

Login API → Generate JWT Token

2. Secured API Access

Authorization: Bearer token

3. Third-Party Login

OAuth → Login with Google

---

9. Security Best Practices ★

What is it

Guidelines for secure API communication and authentication handling.

Key Components

- • Avoid hardcoding credentials
- • Use HTTPS
- • Secure token handling
- • Token expiration management

How YOU should answer

I follow secure authentication practices such as using HTTPS, avoiding hardcoded credentials, and managing tokens securely.

Practical Example

```
Example
Store tokens securely
Use environment variables
```

Common Mistakes

- ✕ Hardcoded passwords
- ✕ Exposed tokens in logs
- ✕ Weak security validation

Expected Interview Questions

- ■ How do you secure API tests?
- ■ Why use HTTPS?
- ■ Best practices for token handling?

---

10. Common Interview Questions ★

- ■ What are request headers?
- ■ Difference between authentication and authorization?
- ■ What is bearer token?
- ■ Difference between API key and token?
- ■ Why OAuth is used?
- ■ Difference between Basic Auth and Bearer Token?

★ Most Important Topics

1. Bearer Token
2. Authentication vs Authorization
3. API Keys vs OAuth
4. Security Best Practices

API VALIDATION

1. What is API Validation

What is it

API validation ensures API responses are correct, complete, secure, and meet business requirements.

Key Components

- Functional validation
- Data validation
- Performance validation
- Security validation

How YOU should answer

API validation ensures that APIs return correct status codes, valid response data, proper headers, and expected behavior under different scenarios.

Practical Example

```
Example
Validate:
- Status Code
- Response Body
- Headers
- Response Time
```

Common Mistakes

- ❌ Validating only status code
- ❌ Ignoring response data
- ❌ No negative testing

Expected Interview Questions

- ■ What is API validation?
- ■ What validations do you perform?
- ■ Why is API validation important?

2. Status Code Validation ★

What is it

Checking whether API returns expected HTTP status codes.

Key Components

- 200 OK
- 201 Created
- 400 Bad Request
- 401 Unauthorized
- 500 Server Error

How YOU should answer

Status code validation confirms whether API operations succeed or fail as expected.

Practical Example

```
Example
expect(response.status()).toBe(200);
```

Common Mistakes

- ✕ Hardcoded incorrect expectations
- ✕ Ignoring failure scenarios

Expected Interview Questions

- ■ Why validate status codes?
- ■ Difference between 200 and 201?
- ■ How do you test failure scenarios?

---

3. Response Body Validation ★

What is it

Validating API response content and business data.

Key Components

- JSON validation
- Field validation
- Data correctness

- Response structure

How YOU should answer

Response body validation ensures APIs return correct business data and expected field values.

Practical Example

```
Example
const body = await response.json();
expect(body.name).toBe('Kalpesh');
```

Common Mistakes

- ✕ Weak assertions
- ✕ Validating only one field
- ✕ Ignoring nested JSON

Expected Interview Questions

- ■ How do you validate response body?
- ■ What validations do you perform?
- ■ How do you validate nested JSON?

---

4. Header Validation

What is it

Validating response headers returned by APIs.

Key Components

- Content-Type
- Authorization
- Cache-Control
- Security headers

How YOU should answer

Header validation ensures APIs return correct metadata such as content type, authorization behavior, and security-related information.

Practical Example

```
Example
expect(response.headers()['content-type'])
```

```
.toContain('application/json');
```

Common Mistakes

- ✕ Ignoring headers validation
- ✕ Incorrect content type expectations

Expected Interview Questions

- ■ Why validate headers?
- ■ Which headers do you validate?
- ■ What is Content-Type?

---

5. Schema Validation ★

What is it

Schema validation verifies API response structure and data types.

Key Components

- • JSON schema
- • Data types
- • Required fields
- • Structure validation

How YOU should answer

Schema validation ensures API responses follow expected structure, field names, and data types consistently.

Practical Example

```
Example
{
"id": "number",
"name": "string"
}
```

Common Mistakes

- ✕ Validating only field values
- ✕ Ignoring missing fields
- ✕ No structure validation

Expected Interview Questions

- What is schema validation?
- Why schema validation is important?
- Difference between field and schema validation?

---

6. Response Time Validation

What is it

Validating API performance and response speed.

Key Components

- Response time
- Performance check
- SLA validation

How YOU should answer

Response time validation ensures APIs respond within acceptable performance limits.

Practical Example

```
Example
expect(responseTime).toBeLessThan(2000);
```

Common Mistakes

- ✕ Ignoring performance validation
- ✕ Unrealistic thresholds

Expected Interview Questions

- Why validate response time?
- Acceptable API response time?
- How do you measure performance?

---

7. Data Validation

What is it

Validating actual business data returned by APIs.

Key Components

- Field values
- Database consistency
- Dynamic data
- Business logic

How YOU should answer

Data validation ensures APIs return correct and expected business information according to requirements.

Practical Example

```
Example
User ID should match created user.
```

Common Mistakes

- ✕ Static validations only
- ✕ Ignoring dynamic values

Expected Interview Questions

- ■ What is data validation?
- ■ How do you validate dynamic data?
- ■ Real-time validation example?

---

8. Negative Validation ★

What is it

Testing API behavior with invalid or unexpected inputs.

Key Components

- Invalid payload
- Missing token
- Invalid endpoint
- Boundary testing

How YOU should answer

Negative validation ensures APIs handle invalid requests gracefully and return meaningful error responses.

Practical Example

```
Example
```

```
Invalid token → 401
Invalid endpoint → 404
```

Common Mistakes

- ✕ Testing only positive scenarios
- ✕ Ignoring invalid data tests

Expected Interview Questions

- ■ What is negative testing?
- ■ Why is negative validation important?
- ■ Examples of negative scenarios?

---

9. Real-Time Scenarios ★

1. Login Validation

Validate: Token generated, Status code 200, Response time

2. User Creation

Validate: User ID generated, 201 Created, Correct response body

3. Invalid Login

Validate: 401 Unauthorized, Error message

---

10. Best Practices ★

What is it

Guidelines for writing reliable API validations.

Key Components

- • Strong assertions
- • Negative testing
- • Schema validation
- • Reusable validations

How YOU should answer

I perform comprehensive API validations including status codes, body, schema, headers, response time, and negative scenarios for reliable automation.

Common Mistakes

- ✕ Weak validations
- ✕ No schema checks
- ✕ Ignoring error responses

Expected Interview Questions

- ■ What validations do you perform?
- ■ How do you validate APIs effectively?
- ■ Why schema validation is important?

★ Most Important Topics

1. Response Body Validation
2. Schema Validation
3. Negative Validation
4. Response Time Validation

API CHAINING

1. What is API Chaining

What is it

API chaining means using data from one API response in another API request.

Key Components

- Dynamic response data
- Sequential requests
- Dependency flow
- Request-response linking

How YOU should answer

API chaining is a process where one API response provides data required for another API request, such as tokens, IDs, or transaction references.

Practical Example

```
Example
Create User API → Get User ID → Fetch User Details API
```

Common Mistakes

- ✕ Hardcoding IDs
- ✕ Ignoring dependency failures
- ✕ Poor response handling

Expected Interview Questions

- ■ What is API chaining?
- ■ Why API chaining is important?
- ■ Real-time example of chaining?

---

2. Why API Chaining is Used ★

What is it

API chaining is used to automate real business workflows where APIs depend on previous responses.

Key Components

- Dynamic workflows
- Data dependency
- End-to-end flow
- Real-time testing

How YOU should answer

API chaining helps simulate real-world workflows by dynamically passing response data between APIs instead of hardcoding values.

Practical Example

```
Example
Login API → Token → Access secured APIs
```

Common Mistakes

- ✕ Using static data
- ✕ Ignoring response validations

Expected Interview Questions

- ■ Why use API chaining?
- ■ Benefits of chaining?
- ■ Why avoid hardcoded data?

---

3. Dynamic Data Handling

What is it

Using runtime-generated values during API execution.

Key Components

- Dynamic IDs
- Tokens
- Session values
- Runtime variables

How YOU should answer

Dynamic data handling improves test reliability by using real-time response values instead of hardcoded test data.

Practical Example

```
Example
const userId = responseBody.id;
```

Common Mistakes

- ✕ Hardcoded test data
- ✕ Reusing expired tokens

Expected Interview Questions

- ■ What is dynamic data?
- ■ Why avoid hardcoded values?
- ■ How do you manage runtime data?

---

4. Token Chaining ★

What is it

Using authentication token generated from one API in subsequent APIs.

Key Components

- • Login API
- • JWT token
- • Authorization header
- • Secured APIs

How YOU should answer

Token chaining is commonly used where login API generates authentication token, and the token is passed to secured APIs through Authorization headers.

Practical Example

```
Example
const token = loginResponse.token;
headers: {
Authorization: `Bearer ${token}`
}
```

Common Mistakes

- ✕ Token expiration handling issues

✕ Incorrect Authorization format

Expected Interview Questions

- What is token chaining?
- How do you handle secured APIs?
- Real-time token flow example?

---

5. ID Chaining ★

What is it

Using resource IDs generated from one API in another API request.

Key Components

- Dynamic resource IDs
- CRUD operations
- Resource linking

How YOU should answer

ID chaining is used when APIs generate dynamic resource identifiers required for future operations like fetch, update, or delete.

Practical Example

```
Example
const userId = createUserResponse.id;
GET /users/{userId}
```

Common Mistakes

✕ Hardcoded IDs

✕ Invalid resource handling

Expected Interview Questions

- What is ID chaining?
- Why dynamic IDs are important?
- Real-time CRUD example?

---

6. Multi-Step API Flow ★

What is it

Executing multiple dependent APIs sequentially to simulate complete workflows.

Key Components

- Sequential execution
- Dependency management
- Workflow automation

How YOU should answer

Multi-step API flows automate complete business scenarios where multiple APIs interact together in sequence.

Practical Example

```
Example
Login → Create User → Fetch User → Update User → Delete User
```

Common Mistakes

- ✕ Ignoring intermediate validations
- ✕ No cleanup handling

Expected Interview Questions

- ■ Explain complete API flow.
- ■ How do you automate end-to-end APIs?
- ■ Challenges in chained APIs?

---

7. Real-Time Scenarios ★

1. E-Commerce Flow

Login → Add Product → Place Order → Payment

2. Banking Flow

Login → Fetch Balance → Transfer Money

3. User Management Flow

Create User → Update User → Delete User

---

8. Common Challenges

What is it

Problems commonly faced during chained API automation.

Key Components

- Token expiration
- Dynamic data issues
- Dependency failures
- Cleanup management

How YOU should answer

Common API chaining challenges include token expiration, dependency failures, invalid dynamic data, and maintaining execution order.

Practical Example

```
Example
Expired Token → 401 Unauthorized
```

Common Mistakes

- ✕ No retry handling
- ✕ Weak cleanup logic

Expected Interview Questions

- ■ Challenges in API chaining?
- ■ How do you handle expired tokens?
- ■ How do you manage dependencies?

---

9. Best Practices ★

What is it

Guidelines for reliable chained API automation.

Key Components

- Dynamic data usage
- Proper validations
- Cleanup handling
- Reusable methods

How YOU should answer

I use dynamic response handling, reusable API utilities, proper validations, and cleanup mechanisms for stable API chaining automation.

Practical Example

```
Example
Store token dynamically
Validate each API response
```

Common Mistakes

- ✕ Hardcoded dependencies
- ✕ No response validation

Expected Interview Questions

- ■ Best practices for API chaining?
- ■ How do you handle reusable flows?
- ■ Why dynamic data is important?

---

10. Common Interview Questions ★

- ■ What is API chaining?
- ■ Why is chaining important?
- ■ What is token chaining?
- ■ Difference between static and dynamic data?
- ■ Real-time chained API example?
- ■ Challenges in API chaining?

★ Most Important Topics

1. Token Chaining
2. Dynamic Data Handling
3. Multi-Step API Flow
4. Real-Time Chaining Scenarios

POSTMAN

1. What is Postman

What is it

Postman is an API testing and development tool used to send requests, validate responses, and automate API workflows.

Key Components

- API requests
- Collections
- Environment variables
- Automation scripts
- API debugging

How YOU should answer

Postman is widely used for API testing, debugging, automation, and validating request-response behavior during development and testing.

Practical Example

```
Example
Send GET request → Validate Response → Save Collection
```

Common Mistakes

- ✕ Hardcoding environment values
- ✕ Ignoring automation capabilities
- ✕ No response validations

Expected Interview Questions

- ■ What is Postman?
- ■ Why Postman is used?
- ■ Have you used Postman in projects?

---

2. Postman Features ★

What is it

Important capabilities provided by Postman.

Key Components

- API testing
- Automation
- Collections
- Environment handling
- Collection runner

How YOU should answer

Postman supports API testing, request management, environment handling, scripting, automation execution, and API collaboration.

Practical Example

```
Example
Collections + Variables + Automation Scripts
```

Common Mistakes

- ✕ Using Postman only for manual testing
- ✕ Ignoring reusable collections

Expected Interview Questions

- ■ Features of Postman?
- ■ Benefits of Postman?
- ■ Why use collections?

---

3. Collections

What is it

Collections are groups of saved API requests.

Key Components

- Organized requests
- Folder structure
- Reusability
- Shared workflows

How YOU should answer

Collections help organize APIs into reusable workflows and simplify execution of multiple API requests together.

Practical Example

```
Example
User APIs
■■■ Login API
■■■ Create User API
■■■ Delete User API
```

Common Mistakes

- ✕ Unorganized collections
- ✕ Duplicate APIs

Expected Interview Questions

- ■ What are collections?
- ■ Benefits of collections?
- ■ How do you organize APIs?

---

4. Environment Variables ★

What is it

Variables used to store reusable dynamic values.

Key Components

- • Base URL
- • Tokens
- • Environment configs
- • Dynamic data

How YOU should answer

Environment variables help avoid hardcoding and allow APIs to run across different environments like QA, UAT, and Production.

Practical Example

```
Example
{{baseUrl}}
```

```
{{token}}
```

Common Mistakes

- ✕ Hardcoding URLs
- ✕ Storing sensitive credentials insecurely

Expected Interview Questions

- ■ What are environment variables?
- ■ Why use variables?
- ■ Benefits of dynamic environments?

---

5. Request Creation

What is it

Creating and configuring API requests in Postman.

Key Components

- • URL
- • HTTP methods
- • Headers
- • Body
- • Authorization

How YOU should answer

Request creation involves configuring API endpoint, method, headers, authentication, and payload for API execution.

Practical Example

```
Example
POST /users
Content-Type: application/json
```

Common Mistakes

- ✕ Wrong HTTP method
- ✕ Invalid JSON payload
- ✕ Missing authentication

Expected Interview Questions

- How do you create requests?
- What request components are required?
- Common request mistakes?

---

6. Response Validation

What is it

Validating API responses after request execution.

Key Components

- Status code
- Response body
- Headers
- Response time

How YOU should answer

Response validation ensures APIs return expected data, status codes, and response structures.

Practical Example

```
Example
pm.response.to.have.status(200);
```

Common Mistakes

- ✕ Weak validations
- ✕ Validating only status code

Expected Interview Questions

- What validations do you perform?
- How do you validate responses?
- Why response validation matters?

---

7. Authorization Handling ★

What is it

Managing API authentication in Postman.

Key Components

- Bearer token
- Basic auth
- OAuth
- API keys

How YOU should answer

Postman supports multiple authentication mechanisms such as bearer tokens, OAuth, API keys, and basic authentication for secured API testing.

Practical Example

```
Example
Authorization → Bearer Token
```

Common Mistakes

- ✕ Expired tokens
- ✕ Hardcoded credentials

Expected Interview Questions

- ■ Which auth methods have you used?
- ■ How do you manage tokens?
- ■ Difference between API key and token?

---

8. Pre-request Scripts

What is it

Scripts executed before API requests.

Key Components

- Dynamic token generation
- Variable setup
- Data preparation

How YOU should answer

Pre-request scripts help prepare dynamic data, tokens, or variables before API execution.

Practical Example

```
Example
pm.environment.set("token", "abc123");
```

Common Mistakes

- ✕ Complex unnecessary scripting
- ✕ Hardcoded values

Expected Interview Questions

- ■ What are pre-request scripts?
- ■ Why are they useful?
- ■ Real-time use case?

---

9. Tests Tab ★

What is it

Section used to write validations and assertions after API execution.

Key Components

- • Assertions
- • Status validation
- • JSON validation
- • Dynamic variables

How YOU should answer

Tests tab is used for writing validations such as status code checks, response body validations, and dynamic data extraction.

Practical Example

```
Example
pm.test("Status code is 200", function () {
pm.response.to.have.status(200);
});
```

Common Mistakes

- ✕ Weak assertions
- ✕ No negative validations

Expected Interview Questions

- ■ What is Tests tab?
- ■ How do you validate APIs in Postman?

■ What assertions have you used?

10. Collection Runner

What is it

Tool used to execute multiple API requests together.

Key Components

• Sequential execution
• Data-driven testing
• Batch execution

How YOU should answer

Collection Runner allows executing complete API workflows and supports data-driven API testing.

Practical Example

```
Example
Run Login → Create User → Delete User
```

Common Mistakes

✕ Ignoring execution order
✕ No data cleanup

Expected Interview Questions

■ What is Collection Runner?
■ Benefits of Collection Runner?
■ Have you done data-driven execution?

11. Newman CLI ★

What is it

Newman is Postman's command-line tool used for automation and CI/CD execution.

Key Components

• CLI execution
• Jenkins integration
• Reporting

- Automation pipelines

How YOU should answer

Newman helps execute Postman collections through command line and supports CI/CD integrations like Jenkins.

Practical Example

```
Example
newman run collection.json
```

Common Mistakes

- ✕ Ignoring reporting
- ✕ Incorrect environment setup

Expected Interview Questions

- ■ What is Newman?
- ■ Why use Newman?
- ■ How do you integrate Postman with Jenkins?

---

12. Real-Time Scenarios ★

1. Login Automation

Generate token → Store variable → Access secured APIs

2. CRUD API Workflow

Create → Fetch → Update → Delete

3. Multi-Environment Testing

QA → UAT → Production

---

13. Best Practices ★

What is it

Guidelines for efficient Postman usage.

Key Components

- Reusable collections
- Environment variables

- Proper assertions
- Secure credential handling

How YOU should answer

I use reusable collections, environment variables, proper validations, and automation-friendly structures for maintainable API testing.

Common Mistakes

- ✕ Hardcoded data
- ✕ Weak assertions
- ✕ No collection organization

Expected Interview Questions

- ■ Best practices in Postman?
- ■ How do you manage reusable APIs?
- ■ How do you automate Postman collections?

★ Most Important Topics

1. Environment Variables
2. Tests Tab
3. Newman CLI
4. Collection Runner

REST ASSURED

1. What is Rest Assured

What is it

Rest Assured is a Java library used for automating REST API testing.

Key Components

- Java-based
- API automation
- Request handling
- Response validation

How YOU should answer

Rest Assured is a Java-based API automation framework widely used for testing REST APIs with readable syntax and strong validation support.

Practical Example

```
Example
given()
.when()
.get("/users")
.then()
.statusCode(200);
```

Common Mistakes

- ✕ Weak assertions
- ✕ Hardcoded data
- ✕ Poor reusable design

Expected Interview Questions

- ■ What is Rest Assured?
- ■ Why Rest Assured is popular?
- ■ Advantages of Rest Assured?

2. Rest Assured Architecture

What is it

Structure and flow of Rest Assured API automation.

Key Components

- Request
- Response
- Validation
- Assertions

How YOU should answer

Rest Assured follows request-response architecture where APIs are executed and validated using fluent Java syntax.

Practical Example

```
Example
given()
.when()
.then()
```

Common Mistakes

- ✕ Mixing validations and request setup
- ✕ No reusable utilities

Expected Interview Questions

- ■ Explain Rest Assured flow.
- ■ What is given-when-then?
- ■ How does Rest Assured work?

---

3. HTTP Methods in Rest Assured ★

What is it

Executing API methods using Rest Assured.

Key Components

- GET

- POST
- PUT
- DELETE
- PATCH

How YOU should answer

Rest Assured supports all HTTP methods for performing CRUD operations and validating API behavior.

Practical Example

```
Example
given()
.when()
.post("/users");
```

Common Mistakes

- ✕ Wrong HTTP method usage
- ✕ Missing validations

Expected Interview Questions

- ■ Which HTTP methods have you automated?
- ■ Difference between POST and PUT?
- ■ Real-time CRUD example?

---

4. Request Specification ★

What is it

Reusable request configuration in Rest Assured.

Key Components

- Base URI
- Headers
- Authentication
- Reusable setup

How YOU should answer

Request specifications help centralize reusable request configurations such as base URL, headers, and authentication.

Practical Example

```
Example
RequestSpecification req =
given()
.baseUri("https://api.com");
```

Common Mistakes

- ✕ Duplicate request setup
- ✕ Hardcoded configurations

Expected Interview Questions

- ■ What is RequestSpecification?
- ■ Why use reusable requests?
- ■ Benefits of centralized setup?

---

5. Response Validation ★

What is it

Validating API responses using assertions.

Key Components

- • Status code
- • Response body
- • Headers
- • Schema validation

How YOU should answer

Response validation ensures APIs return correct status codes, response data, and expected structures.

Practical Example

```
Example
then()
.statusCode(200)
.body("name", equalTo("Kalpesh"));
```

Common Mistakes

- ✕ Weak assertions

❌ Validating only status codes

Expected Interview Questions

- What validations do you perform?
- How do you validate response body?
- How do you validate JSON?

---

6. Authentication Handling

What is it

Managing secured API authentication in Rest Assured.

Key Components

- Bearer token
- Basic auth
- OAuth
- API keys

How YOU should answer

Rest Assured supports multiple authentication mechanisms including bearer tokens, OAuth, API keys, and basic authentication.

Practical Example

```
Example
given()
.auth()
.oauth2(token);
```

Common Mistakes

❌ Hardcoded credentials

❌ Expired token handling issues

Expected Interview Questions

- How do you handle authentication?
- Which auth types have you used?
- Real-time secured API example?

---

7. JSON Path Extraction ★

What is it

Extracting response data dynamically from JSON.

Key Components

- Dynamic extraction
- JSON path
- Runtime values

How YOU should answer

JSON Path is used to extract dynamic response values such as IDs or tokens from API responses.

Practical Example

```
Example
String token =
response.jsonPath().getString("token");
```

Common Mistakes

- ✕ Incorrect JSON path
- ✕ Hardcoded response values

Expected Interview Questions

- ■ What is JSON Path?
- ■ How do you extract dynamic data?
- ■ Real-time extraction example?

---

8. Serialization & Deserialization ★

What is it

Converting Java objects to JSON and vice versa.

Key Components

- POJO classes
- JSON conversion
- Object mapping

How YOU should answer

Serialization converts Java objects into JSON requests, while deserialization converts JSON responses into Java objects.

Practical Example

```
Example
User user = response.as(User.class);
```

Common Mistakes

- ✕ Incorrect POJO mapping
- ✕ Field mismatch issues

Expected Interview Questions

- ■ What is serialization?
- ■ Difference between serialization and deserialization?
- ■ Why use POJO classes?

---

9. API Chaining ★

What is it

Using response data from one API into another API.

Key Components

- • Dynamic IDs
- • Tokens
- • Sequential APIs

How YOU should answer

API chaining improves automation reliability by dynamically passing response data between dependent APIs.

Practical Example

```
Example
String userId =
response.jsonPath().getString("id");
```

Common Mistakes

- ✕ Hardcoded IDs

✕ Ignoring dependency failures

Expected Interview Questions

- What is API chaining?
- Why dynamic data matters?
- Real-time chaining example?

---

10. Framework Integration ★

What is it

Integrating Rest Assured into automation frameworks.

Key Components

- TestNG/JUnit
- Maven
- Reporting
- CI/CD

How YOU should answer

Rest Assured is integrated with frameworks like TestNG, Maven, Jenkins, and reporting tools for scalable API automation.

Practical Example

```
Example
Rest Assured + TestNG + Maven + Jenkins
```

Common Mistakes

✕ Poor framework structure

✕ No reusable utilities

Expected Interview Questions

- How do you integrate Rest Assured?
- Framework design approach?
- CI/CD integration experience?

---

11. Real-Time Scenarios ★

1. Login Automation

Generate token → Access secured APIs

2. CRUD Operations

Create → Fetch → Update → Delete

3. Dynamic API Chaining

Create User → Extract ID → Fetch User

---

12. Best Practices ★

What is it

Guidelines for scalable API automation using Rest Assured.

Key Components

- Reusable utilities
- Strong validations
- Dynamic data handling
- Centralized configurations

How YOU should answer

I use reusable request specifications, strong validations, dynamic data extraction, and framework-level utilities for maintainable API automation.

Common Mistakes

- ✕ Hardcoded test data
- ✕ Weak assertions
- ✕ Duplicate API methods

Expected Interview Questions

- ■ Best practices in Rest Assured?
- ■ How do you maintain API frameworks?
- ■ How do you handle reusable methods?

★ Most Important Topics

1. Request Specification

2. Response Validation

3. Serialization & Deserialization

4. JSON Path Extraction

DATA-DRIVEN API TESTING

1. What is Data-Driven Testing

What is it

Data-driven testing is a testing approach where test data is separated from test scripts and executed using multiple datasets.

Key Components

- External test data
- Reusable scripts
- Multiple datasets
- Parameterization

How YOU should answer

Data-driven testing improves automation scalability by executing the same test logic with different datasets stored externally.

Practical Example

```
Example
Single Login API Test
→ Multiple usernames/passwords
```

Common Mistakes

- ✕ Hardcoded test data
- ✕ Duplicate test scripts
- ✕ Poor data management

Expected Interview Questions

- ■ What is data-driven testing?
- ■ Why use data-driven frameworks?
- ■ Benefits of parameterization?

---

2. Why Data-Driven Testing is Used ★

What is it

Using reusable automation with multiple test datasets.

Key Components

- Scalability
- Reusability
- Reduced duplication
- Faster execution

How YOU should answer

Data-driven testing improves maintainability and allows the same automation flow to validate multiple scenarios efficiently.

Practical Example

```
Example
Login API tested with:
- Valid users
- Invalid users
- Empty credentials
```

Common Mistakes

- ✕ Creating separate tests for every dataset
- ✕ Static test execution

Expected Interview Questions

- ■ Why is data-driven testing important?
- ■ Advantages of reusable data?
- ■ Real-time use cases?

---

3. Types of Test Data

What is it

Different formats used to store automation data.

Key Components

- JSON
- Excel
- CSV
- Database

- Environment variables

How YOU should answer

Test data can be managed using JSON, Excel, CSV, databases, or external configuration files depending on project needs.

Practical Example

```
Example
testData.json
users.xlsx
data.csv
```

Common Mistakes

- ✕ Poor data organization
- ✕ Duplicate datasets

Expected Interview Questions

- ■ What data sources have you used?
- ■ Why JSON is preferred?
- ■ Difference between CSV and JSON?

---

4. JSON Data Handling ★

What is it

Using JSON files to store and manage API test data.

Key Components

- Structured data
- Nested objects
- Dynamic payloads

How YOU should answer

JSON is widely used in API automation because it is lightweight, structured, and matches REST API request-response formats.

Practical Example

```
Example
{
```

```
"username": "Kalpesh",
"password": "12345"
}
```

Common Mistakes

- ✕ Invalid JSON syntax
- ✕ Hardcoded dynamic values

Expected Interview Questions

- ■ Why use JSON for API testing?
- ■ How do you manage test data?
- ■ How do you handle nested JSON?

---

5. Excel Data Handling

What is it

Using Excel sheets for storing automation datasets.

Key Components

- • Rows and columns
- • Bulk test data
- • External data source

How YOU should answer

Excel files are commonly used for managing large datasets and executing repetitive automation scenarios.

Practical Example

```
Example
Username | Password
user1 | pass1
user2 | pass2
```

Common Mistakes

- ✕ Complex Excel dependencies
- ✕ Poor file maintenance

Expected Interview Questions

- ■ Have you used Excel in automation?

- Advantages of Excel datasets?
- Challenges in Excel handling?

---

6. CSV Data Handling

What is it

Using CSV files for lightweight data-driven execution.

Key Components

- Comma-separated values
- Lightweight structure
- Bulk datasets

How YOU should answer

CSV files are lightweight and useful for executing large repetitive datasets efficiently.

Practical Example

```
Example
username,password
user1,pass1
```

Common Mistakes

- ✕ Invalid formatting
- ✕ Encoding issues

Expected Interview Questions

- Difference between CSV and Excel?
- Why use CSV?
- Advantages of lightweight data?

---

7. Dynamic Data Generation ★

What is it

Generating unique runtime test data during execution.

Key Components

- Dynamic emails

- Random values
- Unique IDs
- Runtime variables

How YOU should answer

Dynamic data generation avoids duplicate conflicts and improves automation reliability by creating unique runtime values.

Practical Example

```
Example
const email =
`test${Date.now()}@mail.com`;
```

Common Mistakes

- ✕ Duplicate test data
- ✕ Reusing static users

Expected Interview Questions

- ■ Why dynamic data is important?
- ■ How do you generate runtime values?
- ■ Real-time example?

---

8. Parameterization ★

What is it

Passing multiple input values into reusable test scripts.

Key Components

- Reusable tests
- Dynamic inputs
- Multiple datasets

How YOU should answer

Parameterization improves reusability by allowing the same automation logic to execute with different input values.

Practical Example

```
Example
```

```
Login(username, password)
```

Common Mistakes

- ✕ Hardcoded parameters
- ✕ Poor reusable design

Expected Interview Questions

- ■ What is parameterization?
- ■ Why parameterization matters?
- ■ Real-time parameterized flow?

---

9. Real-Time Scenarios ★

1. Login Testing

Test: Valid users, Invalid users, Locked users

2. Bulk User Creation

Read user data from JSON/Excel

3. Dynamic Registration

Generate unique email IDs

---

10. Framework Integration ★

What is it

Integrating data-driven approach into automation frameworks.

Key Components

- • Utility classes
- • External files
- • Reusable methods
- • Centralized data

How YOU should answer

Data-driven testing is integrated into frameworks using reusable utilities and centralized external test data management.

Practical Example

```
Example
Framework
■■■ testData.json
■■■ ExcelUtils.java
```

Common Mistakes

- ✕ Duplicate data logic
- ✕ No centralized data handling

Expected Interview Questions

- ■ How do you design data-driven frameworks?
- ■ How do you manage reusable test data?
- ■ Framework integration approach?

---

11. Best Practices ★

What is it

Guidelines for scalable data-driven automation.

Key Components

- • Reusable data
- • Dynamic generation
- • Centralized management
- • Clean datasets

How YOU should answer

I use reusable and centralized datasets, dynamic runtime values, and clean parameterized utilities for scalable automation.

Practical Example

```
Example
Use:
- JSON data
- Utility methods
- Dynamic values
```

Common Mistakes

- ✕ Hardcoded datasets
- ✕ Duplicate test cases
- ✕ Poor data cleanup

Expected Interview Questions

- Best practices for data-driven testing?
- Why centralized data matters?
- How do you maintain test data?

---

12. Common Interview Questions ★

- What is data-driven testing?
- Why parameterization is important?
- Difference between JSON and CSV?
- Why use dynamic data?
- Real-time data-driven scenario?
- How do you manage external data?

★ Most Important Topics

1. JSON Data Handling

2. Parameterization

3. Dynamic Data Generation

4. Framework Integration

CI/CD INTEGRATION (JENKINS)

1. What is CI/CD

What is it

CI/CD stands for Continuous Integration and Continuous Deployment/Delivery.

Key Components

- Automated builds
- Automated testing
- Continuous deployment
- Faster feedback

How YOU should answer

CI/CD automates code integration, testing, and deployment processes to improve software quality and delivery speed.

Practical Example

```
Example
Code Commit → Build → Test Execution → Report → Deployment
```

Common Mistakes

- ✕ Manual execution dependency
- ✕ No automated validations

Expected Interview Questions

- ■ What is CI/CD?
- ■ Benefits of CI/CD?
- ■ Why automation is important in CI/CD?

---

2. What is Jenkins ★

What is it

Jenkins is an open-source automation server used for CI/CD pipeline execution.

Key Components

- Pipelines
- Build jobs
- Scheduling
- Plugins
- Reporting

How YOU should answer

Jenkins is widely used for automating build, test execution, reporting, and deployment processes in CI/CD pipelines.

Practical Example

```
Example
Jenkins Job → Execute API Automation Suite
```

Common Mistakes

- ✕ Poor job organization
- ✕ No reporting integration

Expected Interview Questions

- ■ What is Jenkins?
- ■ Why Jenkins is used?
- ■ Have you integrated automation with Jenkins?

---

3. Why Jenkins is Used

What is it

Using Jenkins for automated execution and continuous testing.

Key Components

- Automated execution
- Scheduled jobs
- Faster feedback
- Team collaboration

How YOU should answer

Jenkins helps automate repetitive testing and build processes, enabling faster and reliable software delivery.

Practical Example

```
Example
Daily Regression Execution at 2 AM
```

Common Mistakes

- ✕ Manual test execution dependency
- ✕ No pipeline monitoring

Expected Interview Questions

- ■ Why use Jenkins?
- ■ Benefits of automation pipelines?
- ■ Real-time Jenkins usage?

---

4. Jenkins Pipeline Basics ★

What is it

A Jenkins pipeline defines stages for automated execution workflows.

Key Components

- • Stages
- • Build
- • Test
- • Deploy
- • Reporting

How YOU should answer

Jenkins pipelines automate workflows using stages like build, test execution, reporting, and deployment.

Practical Example

```
Example
pipeline {
stages {
stage('Build') {}
stage('Test') {}
}
}
```

Common Mistakes

- ✕ Complex pipeline design
- ✕ No failure handling

Expected Interview Questions

- ■ What is Jenkins pipeline?
- ■ Explain pipeline stages.
- ■ Difference between freestyle and pipeline jobs?

---

5. Integrating API Automation with Jenkins ★

What is it

Executing API automation frameworks through Jenkins pipelines.

Key Components

- • Maven commands
- • Newman execution
- • Build triggers
- • Reports

How YOU should answer

API automation frameworks are integrated with Jenkins for automated regression execution, scheduled runs, and reporting.

Practical Example

```
Example
Jenkins → Trigger API Automation → Generate Report
```

Common Mistakes

- ✕ No environment configuration
- ✕ Poor report handling

Expected Interview Questions

- ■ How do you integrate automation with Jenkins?
- ■ How are regression suites executed?
- ■ Real-time CI/CD example?

---

6. Maven Integration

What is it

Using Maven commands to execute automation suites in Jenkins.

Key Components

- pom.xml
- Build lifecycle
- Dependency management

How YOU should answer

Maven is integrated with Jenkins to build projects, manage dependencies, and execute automated test suites.

Practical Example

```
Example
mvn clean test
```

Common Mistakes

- ✕ Dependency conflicts
- ✕ Incorrect build configuration

Expected Interview Questions

- ■ How does Maven integrate with Jenkins?
- ■ What Maven commands have you used?
- ■ Why Maven is important?

---

7. Newman Integration

What is it

Executing Postman collections through Jenkins using Newman.

Key Components

- Newman CLI
- Collection execution
- Environment configs
- Reports

How YOU should answer

Newman is used in Jenkins pipelines to automate Postman collection execution and reporting.

Practical Example

```
Example
newman run collection.json
```

Common Mistakes

- ✕ Incorrect environment files
- ✕ No report generation

Expected Interview Questions

- ■ What is Newman?
- ■ How do you run Postman collections in Jenkins?
- ■ Have you automated Postman suites?

---

8. Scheduled Execution

What is it

Running automation suites automatically at scheduled times.

Key Components

- • Cron jobs
- • Nightly execution
- • Regression suites

How YOU should answer

Jenkins supports scheduled automation execution for regression testing using cron-based triggers.

Practical Example

```
Example
Nightly Regression Execution
```

Common Mistakes

- ✕ No monitoring for failures
- ✕ Resource conflicts during execution

Expected Interview Questions

- ■ Have you scheduled automation runs?
- ■ Why nightly execution is important?
- ■ What is cron scheduling?

9. Reporting in Jenkins ★

What is it

Generating and publishing automation reports in Jenkins.

Key Components

- HTML reports
- Allure reports
- Execution logs
- Failure screenshots

How YOU should answer

Jenkins integrates with reporting tools to provide execution summaries, logs, and detailed failure analysis.

Practical Example

```
Example
Execute Tests → Generate Allure Report
```

Common Mistakes

- ✕ No report archiving
- ✕ Poor failure logging

Expected Interview Questions

- ■ Which reports have you used?
- ■ How do you analyze failures?
- ■ Why reporting is important?

10. Real-Time Scenarios ★

1. Nightly Regression

Execute full API regression every night

2. PR Validation

Trigger automation after code commit

3. Multi-Environment Execution

QA → UAT → Staging pipelines

11. Best Practices ★

What is it

Guidelines for stable CI/CD automation execution.

Key Components

- Stable pipelines
- Proper reporting
- Environment handling
- Failure notifications

How YOU should answer

I use stable pipelines, reusable execution commands, proper reporting, and environment management for reliable CI/CD automation.

Common Mistakes

- ✕ Weak failure analysis
- ✕ Hardcoded environment values
- ✕ No notifications setup

Expected Interview Questions

- ■ Best practices in Jenkins automation?
- ■ How do you manage failures?
- ■ How do you maintain CI/CD stability?

---

12. Common Interview Questions ★

- ■ What is Jenkins?
- ■ Explain CI/CD pipeline.
- ■ Difference between freestyle and pipeline jobs?
- ■ How do you integrate API automation with Jenkins?
- ■ What reports have you used?
- ■ Have you scheduled automation runs?

★ Most Important Topics

1. Jenkins Pipeline

2. Automation Integration

3. Reporting in Jenkins

4. Scheduled Execution

SCHEMA VALIDATION

1. What is Schema Validation

What is it

Schema validation verifies whether an API response matches the expected JSON structure and data format.

Key Components

- Response structure
- Field validation
- Data types
- Mandatory fields

How YOU should answer

Schema validation ensures that API responses follow the expected structure, field types, and mandatory field requirements.

Practical Example

```
Example
{
"id": 101,
"name": "Kalpesh"
}

Expected: id → number, name → string
```

Common Mistakes

- ✕ Validating only status code
- ✕ Ignoring response structure
- ✕ Weak field validations

Expected Interview Questions

- ■ What is schema validation?
- ■ Why schema validation is important?
- ■ Difference between field validation and schema validation?

2. Why Schema Validation is Important ★

What is it

Schema validation prevents API contract failures and response structure issues.

Key Components

- API contract validation
- Stability checks
- Response consistency
- Regression safety

How YOU should answer

Schema validation helps detect response structure changes early and ensures API contracts remain stable across releases.

Practical Example

```
Example
Backend changes field: "name" → "fullName"
Schema validation catches it immediately.
```

Common Mistakes

- ✕ Ignoring contract changes
- ✕ Validating only response values

Expected Interview Questions

- ■ Why use schema validation?
- ■ How schema validation improves automation?
- ■ Real-time benefits?

---

3. JSON Schema Basics

What is it

JSON Schema defines expected response structure and rules.

Key Components

- Properties
- Required fields

- Data types
- Arrays
- Objects

How YOU should answer

JSON Schema defines the expected structure, mandatory fields, and data types for API responses.

Practical Example

```
Example
{
"type": "object",
"properties": {
"id": { "type": "integer" }
}
}
```

Common Mistakes

- ✕ Incorrect schema format
- ✕ Missing required validations

Expected Interview Questions

- ■ What is JSON Schema?
- ■ What schema properties have you used?
- ■ Why schemas are important?

---

4. Response Structure Validation

What is it

Validating API response hierarchy and structure.

Key Components

- Parent-child structure
- Nested objects
- Arrays
- Field hierarchy

How YOU should answer

Response structure validation ensures APIs return expected object hierarchy and field organization.

Practical Example

```
Example
{
"user": {
"name": "Kalpesh"
}
}
```

Common Mistakes

- ✕ Ignoring nested structures
- ✕ Partial validation only

Expected Interview Questions

- ■ How do you validate response structure?
- ■ How do you handle nested JSON?
- ■ Why hierarchy validation matters?

---

5. Required Fields Validation ★

What is it

Checking whether mandatory fields exist in response.

Key Components

- • Mandatory keys
- • Required properties
- • Null validations

How YOU should answer

Required field validation ensures critical API response fields are always present and not missing.

Practical Example

```
Example
Required:
- id
- token
- status
```

Common Mistakes

- ✕ Ignoring missing fields
- ✕ No null checks

Expected Interview Questions

- ■ How do you validate required fields?
- ■ Why mandatory fields matter?
- ■ How do you validate null responses?

---

6. Data Type Validation

What is it

Validating response field data types.

Key Components

- • String
- • Integer
- • Boolean
- • Array
- • Object

How YOU should answer

Data type validation ensures APIs return values in expected formats such as string, integer, boolean, or arrays.

Practical Example

```
Example
{
"id": 101,
"active": true
}
```

Common Mistakes

- ✕ Type mismatch issues
- ✕ Ignoring boolean validations

Expected Interview Questions

- ■ Why validate data types?

- Common datatype issues?
- Real-time datatype validation examples?

---

7. Nested JSON Validation ★

What is it

Validating deeply nested response structures.

Key Components

- Nested objects
- Arrays
- Child fields
- JSON hierarchy

How YOU should answer

Nested JSON validation ensures child objects and arrays follow expected structures and formats.

Practical Example

```
Example
{
"user": {
"address": {
"city": "Bangalore"
}
}
}
```

Common Mistakes

- ✕ Weak nested validations
- ✕ Ignoring child objects

Expected Interview Questions

- How do you validate nested JSON?
- Challenges in nested validation?
- Real-time example?

---

8. Schema Validation in Rest Assured ★

What is it

Using Rest Assured libraries to validate JSON schema.

Key Components

- JSON schema validator
- Schema files
- Assertion libraries

How YOU should answer

Rest Assured supports schema validation using JSON schema validator libraries to validate complete response structures.

Practical Example

```
Example
then().assertThat()
.body(matchesJsonSchemaInClasspath("schema.json"));
```

Common Mistakes

- ✕ Incorrect schema file path
- ✕ Weak schema definitions

Expected Interview Questions

- ■ How do you validate schema in Rest Assured?
- ■ Which library have you used?
- ■ Why schema validation is useful?

---

9. Schema Validation in Postman

What is it

Validating response structure using Postman test scripts.

Key Components

- pm.expect
- JSON validation
- Response assertions

How YOU should answer

Postman supports schema and response validation using JavaScript assertions inside the Tests tab.

Practical Example

```
Example
pm.expect(response.id).to.be.a('number');
```

Common Mistakes

- ✕ Weak assertions
- ✕ Missing datatype validations

Expected Interview Questions

- ■ How do you validate schema in Postman?
- ■ What assertions have you used?
- ■ Real-time validation example?

---

10. Real-Time Scenarios ★

1. Login API

Validate: token, userId, response structure

2. User API

Validate: nested address object, phone number array

3. E-Commerce APIs

Validate: order structure, product arrays, payment details

---

11. Best Practices ★

What is it

Guidelines for strong schema validation implementation.

Key Components

- • Reusable schemas
- • Strong assertions
- • Nested validations
- • Contract consistency

How YOU should answer

I use reusable JSON schemas, proper datatype validations, nested structure checks, and strong assertions for stable API validation.

Common Mistakes

- ✕ Weak schema definitions
- ✕ Ignoring nested objects
- ✕ No required field validations

Expected Interview Questions

- ■ Best practices in schema validation?
- ■ How do you maintain schema files?
- ■ Why reusable schemas matter?

---

12. Common Interview Questions ★

- ■ What is schema validation?
- ■ Why schema validation matters?
- ■ Difference between schema and response validation?
- ■ How do you validate nested JSON?
- ■ How do you perform schema validation in Rest Assured?
- ■ Why datatype validation is important?

★ Most Important Topics

1. Required Fields Validation
2. Nested JSON Validation
3. Schema Validation in Rest Assured
4. Response Structure Validation

API MOCKING

1. What is API Mocking

What is it

API mocking simulates API responses without using the actual backend service.

Key Components

- Fake responses
- Mock server
- Request simulation
- Response simulation

How YOU should answer

API mocking helps simulate backend responses when actual APIs are unavailable, unstable, or still under development.

Practical Example

```
Example
Frontend → Mock API → Fake Response
```

Common Mistakes

- ✕ Using outdated mock responses
- ✕ Poor mock maintenance

Expected Interview Questions

- ■ What is API mocking?
- ■ Why API mocking is important?
- ■ Real-time use case of mocking?

---

2. Why API Mocking is Important ★

What is it

Mocking removes dependency on real backend services.

Key Components

- Independent testing
- Faster execution
- Early testing
- Failure simulation

How YOU should answer

API mocking enables independent testing and allows teams to continue automation even when backend APIs are unavailable.

Practical Example

```
Example
Backend not ready
→ Use mock API response
```

Common Mistakes

- ✕ Overusing mocks
- ✕ Not validating against real APIs later

Expected Interview Questions

- ■ Why use mocking?
- ■ Benefits of mock APIs?
- ■ Difference between mock and real APIs?

---

3. Mock Server Basics

What is it

A mock server simulates API endpoints and responses.

Key Components

- Fake endpoints
- Response configuration
- Request handling
- Response delay simulation

How YOU should answer

Mock servers simulate backend APIs by returning predefined responses for testing purposes.

Practical Example

```
Example
GET /users
→ Returns fake user response
```

Common Mistakes

- ✕ Poor endpoint mapping
- ✕ Invalid response structures

Expected Interview Questions

- ■ What is a mock server?
- ■ Why mock servers are used?
- ■ Have you used mock servers?

---

4. Types of Mocking

What is it

Different ways to simulate APIs.

Key Components

- • Static mocking
- • Dynamic mocking
- • Service virtualization
- • Contract mocking

How YOU should answer

API mocking can be static or dynamic depending on whether responses are fixed or generated at runtime.

Practical Example

```
Example
Static:
Fixed JSON response

Dynamic:
Response changes based on request
```

Common Mistakes

- ✕ Unrealistic mock behavior
- ✕ No dynamic validations

Expected Interview Questions

- Types of API mocking?
- Difference between static and dynamic mocks?
- What is service virtualization?

---

5. Mocking in Postman ★

What is it

Using Postman mock servers to simulate APIs.

Key Components

- Mock collections
- Example responses
- Mock URLs

How YOU should answer

Postman supports mock servers that return predefined example responses for API testing and frontend development.

Practical Example

```
Example
Postman Collection
→ Generate Mock Server URL
```

Common Mistakes

- ✕ Incorrect example mapping
- ✕ No response maintenance

Expected Interview Questions

- Have you used Postman mock servers?
- How do mock URLs work?
- Why use Postman mocks?

---

6. Mocking Using WireMock ★

What is it

WireMock is a Java-based tool used for API mocking and service virtualization.

Key Components

- Stub responses
- Dynamic responses
- Request matching
- Response simulation

How YOU should answer

WireMock is widely used for simulating REST APIs and testing systems independently from backend dependencies.

Practical Example

```
Example
stubFor(get(urlEqualTo("/users"))
.willReturn(aResponse()
.withStatus(200)));
```

Common Mistakes

- ✕ Weak request matching
- ✕ Incorrect stub configuration

Expected Interview Questions

- ■ What is WireMock?
- ■ Why use WireMock?
- ■ Real-time WireMock usage?

---

7. Mocking Third-Party APIs

What is it

Simulating external APIs that are unstable, paid, or rate-limited.

Key Components

- Payment APIs
- External integrations
- Dependency isolation

How YOU should answer

Third-party APIs are mocked to avoid dependency issues, cost limitations, and unstable external environments.

Practical Example

```
Example
Payment Gateway API
→ Mock success & failure responses
```

Common Mistakes

- ✕ Ignoring real integration testing
- ✕ Mock mismatch with actual APIs

Expected Interview Questions

- ■ Why mock third-party APIs?
- ■ Challenges with external APIs?
- ■ Real-time example?

---

8. Simulating Error Responses ★

What is it

Testing failure scenarios using mocked error responses.

Key Components

- • 500 errors
- • Timeout simulation
- • Invalid responses
- • Failure handling

How YOU should answer

Mocking helps simulate API failures like timeouts, 500 errors, and invalid responses for negative testing.

Practical Example

```
Example
Simulate:
- 500 Internal Server Error
- Timeout
- Unauthorized response
```

Common Mistakes

- ✕ Only testing success responses

✕ Ignoring negative scenarios

Expected Interview Questions

- How do you test API failures?
- Why simulate timeouts?
- Negative testing examples?

---

9. Real-Time Scenarios ★

1. Backend Not Ready

Frontend team uses mock APIs

2. Payment Gateway Testing

Mock payment success/failure

3. Performance Testing

Simulate delayed API responses

---

10. Best Practices ★

What is it

Guidelines for reliable API mocking.

Key Components

- Realistic responses
- Response maintenance
- Negative scenarios
- Reusable mocks

How YOU should answer

I use realistic mock responses, reusable configurations, and negative scenario simulations for reliable testing.

Practical Example

```
Example
Use:
- Mock servers
- Dynamic responses
```

```
- Failure simulations
```

Common Mistakes

- ✕ Outdated mocks
- ✕ No synchronization with real APIs
- ✕ Ignoring negative cases

Expected Interview Questions

- ■ Best practices in API mocking?
- ■ How do you maintain mock responses?
- ■ Why realistic mocks matter?

---

11. Common Mistakes ★

- ✕ Using unrealistic responses
- ✕ No negative testing
- ✕ Ignoring actual backend validation
- ✕ Poor mock maintenance
- ✕ Hardcoded mock data

---

12. Common Interview Questions ★

- ■ What is API mocking?
- ■ Why use API mocking?
- ■ Difference between mocking and real APIs?
- ■ What is WireMock?
- ■ How do you simulate failures?
- ■ Real-time mocking scenarios?

★ Most Important Topics

1. Mocking in Postman
2. WireMock
3. Simulating Error Responses
4. Mocking Third-Party APIs

REAL-TIME SCENARIOS

1. What are Real-Time Scenarios

What is it

Real-time scenarios are practical business workflows automated using APIs.

Key Components

- End-to-end workflows
- Business validations
- API chaining
- Dynamic data handling

How YOU should answer

Real-time API scenarios simulate actual business workflows such as login, payments, user management, and order processing.

Practical Example

```
Example
Login → Create Order → Payment → Confirmation
```

Common Mistakes

- ✕ Testing isolated APIs only
- ✕ Ignoring business workflows

Expected Interview Questions

- ■ What real-time APIs have you automated?
- ■ Explain one end-to-end flow.
- ■ How do you automate business workflows?

---

2. Login API Automation ★

What is it

Automating user authentication APIs.

Key Components

- Token generation
- Authentication
- Session handling
- Authorization validation

How YOU should answer

Login APIs are automated to validate authentication, token generation, secured access, and authorization flows.

Practical Example

```
Example
Login API
→ Generate JWT Token
→ Access secured APIs
```

Common Mistakes

- ✕ Hardcoded tokens
- ✕ Ignoring token expiration

Expected Interview Questions

- ■ How do you automate login APIs?
- ■ How do you handle tokens?
- ■ What validations do you perform?

---

3. User Management APIs

What is it

Automating CRUD operations for user-related APIs.

Key Components

- Create user
- Fetch user
- Update user
- Delete user

How YOU should answer

User management APIs are automated using CRUD workflows with validations for response data and database consistency.

Practical Example

```
Example
Create User
→ Fetch User
→ Update User
→ Delete User
```

Common Mistakes

- ✕ No cleanup handling
- ✕ Duplicate user creation

Expected Interview Questions

- ■ Have you automated CRUD APIs?
- ■ How do you validate user APIs?
- ■ Real-time CRUD example?

---

4. E-Commerce API Flow ★

What is it

Automating shopping and order management workflows.

Key Components

- • Product APIs
- • Cart APIs
- • Order APIs
- • Payment APIs

How YOU should answer

E-commerce API automation validates product search, cart management, order creation, payment, and order confirmation workflows.

Practical Example

```
Example
Login
→ Add Product
→ Add to Cart
→ Place Order
```

```
→ Payment
```

Common Mistakes

- ✕ Weak order validations
- ✕ Ignoring inventory checks

Expected Interview Questions

- ■ Explain e-commerce API flow.
- ■ How do you automate order APIs?
- ■ What validations are important?

---

5. Banking API Flow

What is it

Automating banking transaction APIs.

Key Components

- • Account APIs
- • Balance APIs
- • Fund transfer APIs
- • Transaction validation

How YOU should answer

Banking APIs are automated to validate account operations, balance updates, transaction processing, and secure authorization.

Practical Example

```
Example
Login
→ Fetch Balance
→ Transfer Funds
→ Validate Transaction
```

Common Mistakes

- ✕ Weak security validations
- ✕ Ignoring transaction consistency

Expected Interview Questions

- Have you automated transaction APIs?
- How do you validate transfers?
- Security challenges in banking APIs?

---

6. Payment Gateway APIs ★

What is it

Testing payment processing APIs and transaction handling.

Key Components

- Payment requests
- Success/failure flows
- Refund APIs
- Gateway integrations

How YOU should answer

Payment gateway APIs are automated to validate transaction processing, payment success/failure handling, and refund workflows.

Practical Example

```
Example
Initiate Payment
→ Payment Success
→ Generate Receipt
```

Common Mistakes

- ✕ No negative testing
- ✕ Ignoring failed transaction handling

Expected Interview Questions

- How do you automate payment APIs?
- How do you handle payment failures?
- Real-time payment scenario?

---

7. File Upload APIs

What is it

Testing APIs that upload documents or files.

Key Components

- Multipart requests
- File validation
- Upload status
- File type checks

How YOU should answer

File upload APIs validate multipart requests, file formats, upload status, and backend storage behavior.

Practical Example

```
Example
Upload PDF
→ Validate upload response
```

Common Mistakes

- ✕ Invalid file handling
- ✕ No size validations

Expected Interview Questions

- ■ Have you automated file upload APIs?
- ■ What validations are important?
- ■ How do you test multipart APIs?

---

8. OTP & Authentication APIs

What is it

Automating OTP generation and verification APIs.

Key Components

- OTP generation
- Expiration validation
- Verification APIs
- Retry handling

How YOU should answer

OTP APIs are automated to validate OTP generation, expiration handling, verification flows, and security behavior.

Practical Example

```
Example
Generate OTP
→ Verify OTP
→ Login Success
```

Common Mistakes

- ✕ Ignoring OTP expiration
- ✕ Weak security testing

Expected Interview Questions

- ■ How do you automate OTP APIs?
- ■ How do you validate expiration?
- ■ Security considerations?

---

9. Microservices API Testing ★

What is it

Testing APIs across distributed microservices architecture.

Key Components

- • Service communication
- • API dependencies
- • Contract validation
- • Distributed workflows

How YOU should answer

Microservices API testing validates communication and data flow between independent services in distributed systems.

Practical Example

```
Example
Order Service
→ Payment Service
```

```
→ Notification Service
```

Common Mistakes

- ✕ Ignoring service dependencies
- ✕ Weak integration validations

Expected Interview Questions

- ■ What challenges exist in microservices testing?
- ■ How do you test service communication?
- ■ Real-time microservices example?

---

10. Error Handling Scenarios

What is it

Testing APIs under failure and invalid conditions.

Key Components

- • 400 errors
- • 401 errors
- • 500 errors
- • Timeout handling

How YOU should answer

Error handling scenarios validate API stability and proper error responses under invalid or failure conditions.

Practical Example

```
Example
Invalid Token
→ 401 Unauthorized
```

Common Mistakes

- ✕ Testing only success flows
- ✕ Weak negative validations

Expected Interview Questions

- ■ How do you test API failures?
- ■ What negative scenarios have you automated?

■ Why negative testing matters?

11. Best Practices ★

What is it

Guidelines for scalable real-time API automation.

Key Components

- Dynamic data
- End-to-end validation
- Reusable workflows
- Strong assertions

How YOU should answer

I use reusable workflows, dynamic test data, end-to-end validations, and proper cleanup mechanisms for stable API automation.

Common Mistakes

✕ Hardcoded workflows
✕ Weak business validations
✕ No cleanup handling

Expected Interview Questions

■ Best practices in API automation?
■ How do you automate real-time workflows?
■ How do you maintain stable automation?

12. Common Interview Questions ★

■ What real-time APIs have you automated?
■ Explain one complete API workflow.
■ How do you automate payment APIs?
■ What challenges have you faced?
■ How do you handle dynamic data?
■ How do you automate microservices?

★ Most Important Topics

1. Login API Automation

2. E-Commerce API Flow

3. Payment Gateway APIs

4. Microservices API Testing

BEST PRACTICES

1. What are API Automation Best Practices

What is it

Best practices are standard guidelines used to build stable, scalable, and maintainable API automation frameworks.

Key Components

- Reusability
- Stability
- Scalability
- Maintainability

How YOU should answer

API automation best practices help improve framework stability, maintainability, execution reliability, and scalability.

Practical Example

```
Example
Reusable utilities
+ Dynamic data
+ Strong validations
```

Common Mistakes

- ✕ Hardcoded data
- ✕ Duplicate methods
- ✕ Weak validations

Expected Interview Questions

- ■ What are API automation best practices?
- ■ Why framework standards matter?
- ■ How do you maintain scalable automation?

---

2. Reusable Framework Design ★

What is it

Designing automation frameworks using reusable components.

Key Components

- Utility methods
- Base classes
- Reusable requests
- Centralized configs

How YOU should answer

Reusable framework design reduces code duplication and improves maintainability using shared utilities and centralized configurations.

Practical Example

```
Example
Framework
■■■ BaseAPI.java
■■■ RequestUtils.java
■■■ ConfigManager.java
```

Common Mistakes

- ✕ Duplicate code
- ✕ Poor folder structure

Expected Interview Questions

- ■ How do you design API frameworks?
- ■ Why reusable methods matter?
- ■ Benefits of centralized configs?

---

3. Strong Assertions ★

What is it

Using proper validations for APIs instead of weak checks.

Key Components

- Status validation
- Schema validation
- Response body validation

- Header validation

How YOU should answer

Strong assertions ensure APIs are validated comprehensively using status codes, response data, schema checks, and headers.

Practical Example

```
Example
then()
.statusCode(200)
.body("status", equalTo("success"));
```

Common Mistakes

- ✕ Validating only status code
- ✕ Weak assertions

Expected Interview Questions

- ■ What validations do you perform?
- ■ Why strong assertions matter?
- ■ Difference between weak and strong validations?

---

4. Dynamic Test Data Handling

What is it

Using runtime-generated or externalized data during execution.

Key Components

- JSON files
- Dynamic emails
- Random IDs
- External datasets

How YOU should answer

Dynamic test data handling improves automation reliability and avoids duplicate conflicts using runtime-generated values.

Practical Example

```
Example
```

```
const email =
`test${Date.now()}@mail.com`;
```

Common Mistakes

- ✕ Hardcoded users
- ✕ Duplicate test data

Expected Interview Questions

- ■ Why dynamic data matters?
- ■ How do you manage test data?
- ■ Real-time dynamic data example?

---

5. Proper API Validation

What is it

Performing complete response validation.

Key Components

- • Status codes
- • Response body
- • Headers
- • Response time
- • Schema validation

How YOU should answer

Proper API validation ensures APIs are validated functionally, structurally, and performance-wise.

Practical Example

```
Example
Validate:
- Status
- Schema
- Response time
```

Common Mistakes

- ✕ Partial validation only
- ✕ Ignoring headers

Expected Interview Questions

- What validations do you perform?
- Why schema validation matters?
- How do you validate APIs completely?

---

6. Negative Testing ★

What is it

Testing APIs using invalid or failure scenarios.

Key Components

- Invalid payloads
- Unauthorized access
- Missing fields
- Failure responses

How YOU should answer

Negative testing validates API stability and proper error handling under invalid conditions.

Practical Example

```
Example

Invalid Token

→ 401 Unauthorized
```

Common Mistakes

- ✕ Testing only happy paths
- ✕ Weak failure validations

Expected Interview Questions

- Why negative testing matters?
- What failure scenarios have you tested?
- Real-time negative example?

---

7. Environment Management

What is it

Managing different execution environments.

Key Components

- QA
- UAT
- Staging
- Production configs

How YOU should answer

Environment management helps execute automation across multiple environments using configurable setup files.

Practical Example

```
Example
config.qa.json
config.uat.json
```

Common Mistakes

- ✕ Hardcoded URLs
- ✕ Poor config management

Expected Interview Questions

- ■ How do you manage environments?
- ■ Why centralized configs matter?
- ■ Multi-environment execution example?

---

8. Logging & Reporting

What is it

Capturing execution logs and automation reports.

Key Components

- Logs
- Allure reports
- Execution details
- Failure analysis

How YOU should answer

Logging and reporting help analyze failures, improve debugging, and provide execution visibility.

Practical Example

```
Example
Generate:
- Execution logs
- Allure reports
```

Common Mistakes

- ✕ Weak logs
- ✕ No failure screenshots/reports

Expected Interview Questions

- ■ Which reporting tools have you used?
- ■ Why logs are important?
- ■ How do you debug failures?

---

9. CI/CD Integration

What is it

Integrating automation execution into deployment pipelines.

Key Components

- • Jenkins
- • Maven
- • Scheduled execution
- • Automated reports

How YOU should answer

CI/CD integration enables automated API execution through Jenkins pipelines with reporting and scheduled regression runs.

Practical Example

```
Example
Jenkins
→ Execute API Suite
→ Generate Report
```

Common Mistakes

- ✕ Manual execution dependency
- ✕ No report integration

Expected Interview Questions

- ■ How do you integrate automation into CI/CD?
- ■ Why CI/CD matters?
- ■ Real-time Jenkins usage?

---

10. Error Handling & Retry Logic ★

What is it

Managing unstable APIs and retrying temporary failures.

Key Components

- • Retry mechanisms
- • Timeout handling
- • Failure recovery
- • Exception handling

How YOU should answer

Error handling and retry logic improve automation stability by managing temporary failures and unstable API behavior.

Practical Example

```
Example
Retry API call
if response = 503
```

Common Mistakes

- ✕ Infinite retries
- ✕ Weak exception handling

Expected Interview Questions

- ■ Why retry logic is important?
- ■ How do you handle flaky APIs?
- ■ What timeout strategies have you used?

---

11. Test Data Cleanup

What is it

Removing or resetting created test data after execution.

Key Components

- Delete APIs
- Database cleanup
- Data reset
- Environment stability

How YOU should answer

Test data cleanup prevents duplicate conflicts and maintains stable test environments.

Practical Example

```
Example
Create User
→ Execute Test
→ Delete User
```

Common Mistakes

- ✕ Leaving stale test data
- ✕ No cleanup automation

Expected Interview Questions

- ■ Why cleanup is important?
- ■ How do you maintain stable environments?
- ■ Real-time cleanup example?

---

12. Common Interview Questions ★

- ■ What are API automation best practices?
- ■ How do you design scalable frameworks?
- ■ Why negative testing matters?
- ■ How do you manage dynamic test data?
- ■ Why retry logic is important?
- ■ How do you maintain stable automation?

★ Most Important Topics

1. Reusable Framework Design
2. Strong Assertions
3. Negative Testing
4. Error Handling & Retry Logic